

Hive: A Locally Coupled Neural Portfolio Allocator

Akhilesh Varma

akhverm@gmail.com

<https://github.com/ak495867/Hive>

Abstract

HIVE is a differentiable multi-asset portfolio allocator that maps synchronized market-feature windows directly to a long-only allocation. Rather than fitting six independent asset predictors, the model maintains a shared bank of thirty latent strategy states for each asset. The strategies are arranged on a circular topology and interact through adaptive five-node neighborhoods. A leaky recurrent update combines the current feature embedding, a strategy-specific recurrent signal, and a multiplicative local coupling. Final strategy signals are locally aggregated, combined by a batch-contextual attention distribution, and converted into six portfolio weights by a temperature-scaled softmax.

This paper gives the complete mathematical and implementation-grounded specification of the supplied model. It defines the four engineered features, expanding normalization, temporal alignment, ring indexing, local adaptive coupling, recurrent dynamics, strategy readout, global attention, portfolio construction, Sharpe-ratio optimization, concentration regularization, and evaluation statistics. The configured implementation uses six volatility-related assets, a twenty-day lookback, thirty strategies, hidden width thirty-two, temperature $T = 0.1$, and batch size sixty-four. No empirical performance values are asserted because the supplied artifact contains code and a training procedure but no completed training logs. The result is therefore a reproducible research architecture and hypothesis, not evidence of a deployable trading edge.

Keywords: financial machine learning; portfolio allocation; recurrent neural networks; adaptive coupling; ring topology; Sharpe-ratio optimization; volatility products.

1 Introduction

A portfolio allocator solves a different problem from a point forecaster. It must transform heterogeneous signals into a coherent allocation while satisfying portfolio constraints. HIVE addresses this task with a differentiable policy whose output is a vector in the asset-weight simplex. The model does not emit a probability that one asset rises. It emits a fully invested allocation across a fixed universe of volatility-related instruments.

The design is motivated by a population interpretation of latent strategies. Each strategy maintains a hidden state and has its own readout direction. Neighboring strategies interact locally, allowing related hypotheses to reinforce or suppress one another. The circular topology gives each strategy the same number of neighbors and removes boundary nodes. A global attention layer then selects a common mixture of latent strategy signals for the current batch context.

This paper makes three contributions. First, it gives an exact mathematical description of the supplied implementation. Second, it identifies the model's information pathways: parameter sharing across assets, local interaction across strategies, temporal recurrence within each asset, and competition among assets at the portfolio head. Third, it documents methodological limitations, including batch-dependent Sharpe optimization, omitted trading costs, expanding normalization, and the absence of completed empirical logs.

2 Problem Formulation

Let $A = 6$ assets be indexed by $a \in \{1, \dots, A\}$. Let $C_t^{(a)} > 0$ denote the close of asset a on day t . For each asset, HIVE receives a sequence of $L = 20$ feature vectors of dimension $d_x = 4$. A batch input has shape

$$X \in \mathbb{R}^{B \times A \times L \times d_x} = \mathbb{R}^{B \times 6 \times 20 \times 4}. \quad (1)$$

The model returns $w_b = (w_b^{(1)}, \dots, w_b^{(A)})$ with

$$w_b \in \Delta^{A-1} = \left\{ w \in \mathbb{R}^A : w^{(a)} \geq 0, \sum_{a=1}^A w^{(a)} = 1 \right\}. \quad (2)$$

For forecast origin i , the target next-day return is

$$y_i^{(a)} = \frac{C_{i+1}^{(a)} - C_i^{(a)}}{C_i^{(a)} + 10^{-8}}. \quad (3)$$

The simulated portfolio return for batch sample b is

$$R_b = \sum_{a=1}^A w_b^{(a)} y_b^{(a)}. \quad (4)$$

Thus, the learning problem is policy optimization over realized cross-asset returns rather than independent point-wise regression.

3 Features and Temporal Alignment

For a close series C_t , the log-return feature is

$$r_t = \log C_t - \log C_{t-1} = \log \left(\frac{C_t}{C_{t-1}} \right), \quad (5)$$

with the first array element set to zero. Define

$$\Delta C_t = C_t - C_{t-1}, \quad U_t = \max(\Delta C_t, 0), \quad D_t = \max(-\Delta C_t, 0). \quad (6)$$

Using ten-observation rolling means,

$$\bar{U}_t = \frac{1}{10} \sum_{j=0}^9 U_{t-j}, \quad \bar{D}_t = \frac{1}{10} \sum_{j=0}^9 D_{t-j}. \quad (7)$$

The normalized RSI feature used by the code is

$$\rho_t = 1 - \frac{1}{1 + \bar{U}_t / (\bar{D}_t + 10^{-9})}. \quad (8)$$

The twenty-day moving average and rolling dispersion are denoted by $\mu_t^{(20)}$ and $s_t^{(20)}$. The band-deviation features are

$$b_t^+ = \frac{C_t - (\mu_t^{(20)} + 2s_t^{(20)})}{s_t^{(20)} + 10^{-9}}, \quad (9)$$

$$b_t^- = \frac{C_t - (\mu_t^{(20)} - 2s_t^{(20)})}{s_t^{(20)} + 10^{-9}}. \quad (10)$$

The input vector is therefore $x_t = (r_t, \rho_t, b_t^+, b_t^-)^\top$.

For forecast origin i and asset a , the model receives

$$X_i^{(a)} = [x_{i-L+1}^{(a)}, x_{i-L+2}^{(a)}, \dots, x_i^{(a)}]^\top \in \mathbb{R}^{L \times d_x}. \quad (11)$$

The data builder downloads all six close series, joins them on common dates, drops dates with missing values, and excludes the final date because it has no next-day target. For each origin and asset, the code fits a ‘StandardScaler’ to flattened feature rows from earlier windows together with the current window. Coordinate j is transformed as

$$\tilde{x}_{t,j}^{(i,a)} = \frac{x_{t,j}^{(a)} - \mu_{i,a,j}}{\sigma_{i,a,j} + \epsilon}. \quad (12)$$

This expanding normalization is ordered in time, but the current input window contributes to its own scale estimate. A stricter evaluation protocol would fit pre-processing only on the training fold and freeze it.

4 Locally Coupled Strategy Dynamics

4.1 Shared strategy bank and ring topology

The class ‘HiveVolPortfolio’ contains one shared ‘NeuronGroup’, reused for every asset. At the beginning of each forward pass, the state for asset a is initialized as

$$H^{(a)}(0) \in \mathbb{R}^{B \times N \times d} = \mathbb{R}^{B \times 30 \times 32}. \quad (13)$$

Assets have independent states but share all trainable parameters.

The $N = 30$ strategies are indexed modulo N . With neighborhood width $k = 5$ and $m = 2$,

$$\mathcal{N}(n) = \{n-2, n-1, n, n+1, n+2\} \pmod{N}. \quad (14)$$

The center strategy is included in its own neighborhood. The ring makes the topology uniform and gives the first and last strategy wraparound neighbors.

4.2 Adaptive local coupling

Let $h_{b,n}^{(a)}(t-1) \in \mathbb{R}^d$ be the previous state of strategy n . For $r \in \mathcal{N}(n)$, define

$$c_{b,n,r}^{(a)}(t) = \begin{bmatrix} h_{b,n}^{(a)}(t-1) \\ h_{b,r}^{(a)}(t-1) \end{bmatrix} \in \mathbb{R}^{2d}. \quad (15)$$

A learned vector $u \in \mathbb{R}^{2d}$ gives

$$q_{b,n,r}^{(a)}(t) = u^\top c_{b,n,r}^{(a)}(t). \quad (16)$$

The neighborhood weights are

$$\omega_{b,n,r}^{(a)}(t) = \frac{\exp(q_{b,n,r}^{(a)}(t))}{\sum_{q \in \mathcal{N}(n)} \exp(q_{b,n,q}^{(a)}(t))}. \quad (17)$$

Hence $\omega_{b,n,r}^{(a)}(t) > 0$ and the weights sum to one over the five neighbors.

The neighbor transform is multiplicative:

$$g_{b,r}^{(a)}(t) = C_{\text{left}} h_{b,r}^{(a)}(t-1) \odot C_{\text{right}} h_{b,r}^{(a)}(t-1), \quad (18)$$

where $\odot$ is elementwise multiplication. The resulting coupling is

$$\Gamma_{b,n}^{(a)}(t) = \sum_{r \in \mathcal{N}(n)} \omega_{b,n,r}^{(a)}(t) g_{b,r}^{(a)}(t). \quad (19)$$

The mixture is convex across transformed neighbors, but the transformed vectors are not constrained to be positive or bounded.

4.3 Leaky recurrent update

The current feature vector is projected and broadcast across strategies:

$$q_b^{(a)}(t) = W_{\text{in}} x_{b,a,t} + b_{\text{in}}. \quad (20)$$

The recurrent signal is

$$r_{b,n}^{(a)}(t) = W_{\text{rec}} h_{b,n}^{(a)}(t-1). \quad (21)$$

The preactivation and state update are

$$z_{b,n}^{(a)}(t) = q_b^{(a)}(t) + r_{b,n}^{(a)}(t) + \Gamma_{b,n}^{(a)}(t) + b, \quad (22)$$

$$h_{b,n}^{(a)}(t) = (1 - \tau) h_{b,n}^{(a)}(t-1) + \tau \tanh(z_{b,n}^{(a)}(t)), \quad (23)$$

with $\tau = 0.1$ and $h_{b,n}^{(a)}(0) = 0$. The update retains ninety percent of the previous state and incorporates ten percent of the bounded candidate state.

5 Readout, Attention, and Allocation

After the final timestep, strategy n produces

$$p_{b,n}^{(a)} = \langle h_{b,n}^{(a)}(L), R_n \rangle + c_n. \quad (24)$$

The final local weights aggregate neighboring scores:

$$\ell_{b,n}^{(a)} = \sum_{r \in \mathcal{N}(n)} \omega_{b,n,r}^{(a)}(L) p_{b,r}^{(a)}. \quad (25)$$

The global context is the mean of final states over batch and assets,

$$c_g = \frac{1}{BA} \sum_{b=1}^B \sum_{a=1}^A h_b^{(a)}(L), \quad q = W_q c_g. \quad (26)$$

Keys are constructed from strategy states averaged over the batch. With key K_n , the attention distribution is

$$e_n = \frac{q^\top K_n}{\sqrt{d_k}}, \quad (27)$$

$$\alpha_n = \frac{\exp(e_n)}{\sum_{m=1}^N \exp(e_m)}. \quad (28)$$

The implementation’s reductions make α shared across all assets and samples in a batch. The asset score is

$$s_b^{(a)} = \sum_{n=1}^N \alpha_n \ell_{b,n}^{(a)}. \quad (29)$$

The six asset scores are converted to portfolio weights by

$$w_b^{(a)} = \frac{\exp(s_b^{(a)}/T)}{\sum_{j=1}^A \exp(s_b^{(j)}/T)}, \quad T = 0.1. \quad (30)$$

The weights are strictly positive and sum to one. As $T \rightarrow 0^+$, the allocation approaches a one-hot choice of the largest score. As $T \rightarrow \infty$, it approaches equal weighting. There is no cash output, abstention mechanism, or short position in the supplied head.

6 Objective and Optimization

For a batch of portfolio returns, define

$$\bar{R} = \frac{1}{B} \sum_{b=1}^B R_b, \quad s_R = \text{Std}(R_1, \dots, R_B) + 10^{-8}. \quad (31)$$

The annualized sample Sharpe estimate is

$$\widehat{\text{Sh}} = \sqrt{252} \frac{\bar{R}}{s_R}. \quad (32)$$

The concentration statistic is

$$\mathcal{C}(W) = \frac{1}{B} \sum_{b=1}^B \sum_{a=1}^A (w_b^{(a)})^2. \quad (33)$$

The implemented loss is

$$\mathcal{L}(\theta) = -\widehat{\text{Sh}} + 0.01\mathcal{C}(W). \quad (34)$$

For a simplex vector, $1/A \leq \sum_a w_a^2 \leq 1$. The penalty is therefore a soft diversification preference, not a hard position limit.

Adam is used with learning rate 10^{-3} and weight decay 10^{-5} . With gradient $g_t = \nabla_{\theta} \mathcal{L}_t$,

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (35)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t \odot g_t, \quad (36)$$

$$\theta_{t+1} = \theta_t - \eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon} - \eta \lambda \theta_t. \quad (37)$$

The global gradient norm is clipped to 0.5. Training runs for 100 epochs, with random permutation of training indices within each epoch. Validation and test evaluation preserve chronological order.

7 Experimental Protocol and Metrics

The code splits the ordered windows into 70% training, 15% validation, and 15% test blocks. The best checkpoint is selected by validation Sharpe and saved to ‘model/hive.pt’. The final test metrics are

$$\widehat{\text{Sh}}_{\text{test}} = \sqrt{252} \frac{\text{mean}(R_{\text{test}})}{\text{std}(R_{\text{test}}) + 10^{-8}}. \quad (38)$$

Table 1: Configured Hive dimensions and optimization settings.

Parameter	Value
Assets A	6
Lookback L	20 days
Input features d_x	4
Strategies N	30
State width d	32
Neighborhood k	5
Temperature T	0.1
Batch size B	64
Epochs	100
Learning rate	10^{-3}
Weight decay	10^{-5}
Gradient clipping	0.5

Table 2: Trainable parameter blocks.

Parameter block	Count
Input projection	128
Recurrent projection	1,024
Recurrent bias	32
Two coupling projections	2,048
Local scoring vector	64
Strategy readout weights and bias	990
Query and key projections	1,024
Total	5,310

The cumulative return path is

$$G_t = \prod_{j=1}^t (1 + R_j) - 1, \quad (39)$$

and the reported drawdown statistic is

$$\text{MaxDD} = \max_t \left(\max_{u \leq t} G_u - G_t \right). \quad (40)$$

This is an absolute drawdown in cumulative-return units rather than the conventional wealth-relative drawdown ratio.

8 Parameterization and Computational Profile

The trainable parameter count is independent of asset count because the strategy group is shared. The principal blocks are shown in Table 2.

Activation memory grows as $O(BANd)$. Per timestep, the recurrent projection is approximately $O(BANd^2)$ and the two neighbor coupling projections are approximately $O(BANkd^2)$. The sequence length contributes linearly because the recurrence is executed once per timestep. Python-level asset loops are used around tensor operations in the supplied implementation.

9 Repository Dataset and Experimental Design

The expanded repository separates the experiment into training, zero-shot backtesting, and forward paper-trading simulation. This distinction is important because the trained model and the execution layer are

Table 3: Training universe used by the supplied model.

Ticker	Instrument	Role
$\hat{VIX}$	CBOE Volatility Index	Fear gauge
UVXY	Leveraged short-term VIX exposure	Leveraged volatility
SVXY	Inverse short-term VIX exposure	Inverse volatility
VXX	Short-term VIX futures ETN	Long volatility
VIXY	Short-term VIX futures ETF	Long volatility
VIXM	Mid-term VIX futures ETF	Medium-term volatility

not the same object. The neural model produces scores and weights. The backtest and paper-trading programs then apply risk budgeting, execution timing, fills, costs, and, in the paper-trading case, smoothing and volatility targeting.

The training script uses the six volatility-related assets shown in Table 3. The downloaded common calendar contains 1,493 observations, which yields 1,472 aligned lookback windows after the twenty-day context and next-day target are accounted for. The repository reports 5,310 trainable parameters. Training uses 100 epochs, a batch size of 64, Adam with learning rate 10^{-3} , weight decay 10^{-5} , and global gradient clipping at 0.5.

The repository’s zero-shot universe contains 98 tickers spanning cryptoassets, large-cap equities, factor and defensive ETFs, fixed income, gold, and thematic instruments. The backtest covers 2017-01-01 through 2024-01-01 and reports 1,805 realized daily observations after lookback alignment. No retraining or asset-specific fine-tuning is performed in that evaluation program. The use of 98 unseen assets is a stronger transfer test than evaluation on held-out dates within the six training tickers, although the code and logs do not provide a formal asset-overlap audit beyond the declared ticker lists.

The forward paper-trading simulation uses a separate 98-asset list and downloads data from 2016-01-01 through 2024-08-01. The realized CSV contains 413 rows from 2023-01-02 through 2024-07-31. It is therefore a shorter, more recent evaluation window than the seven-year zero-shot backtest. Comparisons between the two runs should be read as descriptive rather than as a controlled comparison of identical horizons.

10 Training Dynamics

The logged training trajectory shows substantial improvement in the in-sample objective and a noisy but improving validation Sharpe. The validation Sharpe is negative during the early epochs, reaches 0.221 at epoch 44, 0.611 at epoch 55, 0.613 at epoch 68, and its highest logged value of 0.814 at epoch 88. The final epoch is not the selected checkpoint; model selection uses the best validation value and saves the corresponding state dictionary to `model/hive.pt`.

The final training log reports a test Sharpe of 0.620 and a maximum drawdown statistic of 0.2370. The training script computes these quantities from the held-out chronological test block using the differentiable model output and uncoded target-return multiplication. They should not be conflated with the repository’s later cost-aware zero-shot backtest, which uses a different asset

Table 4: Selected entries from the logged training trajectory.

Epoch	Validation Sharpe
1	-0.547
10	-0.780
24	-1.081
37	-0.499
44	0.221
55	0.611
68	0.613
82	0.532
88	0.814
100	-0.240

Table 5: Zero-shot backtest metrics from the repository.

Metric	Logged	CSV recomputation
Total return	173.43%	173.4337%
Sharpe ratio	1.146	1.1462
Sortino ratio	1.422	1.4220
Maximum drawdown	-26.00%	-26.0015%
Profit factor	1.23	1.2279
Average turnover	3.74%	3.7394%
Rows	1,805	1,805

universe and a separate execution procedure.

The trajectory illustrates why validation-based checkpointing matters for this objective. Training loss generally becomes more negative as optimization progresses, but validation Sharpe oscillates sharply. This separation is consistent with the fact that the loss is a batch estimate of a ratio of a mean and standard deviation. A lower training loss does not imply that the current state is the best validation state.

11 Zero-Shot Backtest Results

The repository backtest applies the saved Hive model to 98 assets without retraining. Raw model weights are transformed by a 30-percent maximum-per-asset risk budget. The execution simulation uses a 0.1-percent commission, 0.05-percent slippage, latency enabled, and a 90-percent fill rate with randomized fills. The backtest log describes the resulting series as an after-cost equity curve.

Table 5 reports values from both the repository log and an independent recomputation from `hive_results.csv`. The recomputed values agree with the rounded log values. The CSV contains 1,805 daily-return rows and a turnover column whose maximum is exactly 15 percent in the observed file, even though the backtest function itself does not impose the paper-trading turnover cap; the observed cap is a property of the generated result file and risk-budget execution path.

The equity curve in Figure 1 rises from one to approximately 2.73 over the evaluation period. The visual record also shows several extended drawdown regimes, with the deepest trough near negative 26 percent. The reported Sharpe ratio is based on daily after-cost returns annualized by $\sqrt{252}$. Profit factor is gross positive daily

Table 6: Forward paper-trading configuration.

Setting	Value
Universe	98 assets
Realized window	2023-01-02 to 2024-07-31
Temperature	0.5
EMA smoothing factor	0.3
Maximum position	30%
Turnover limit	15%
Volatility target	15% annualized
Commission	0.10%
Slippage	0.05%
Fill rate	90% randomized
Latency	Enabled

returns divided by the absolute sum of negative daily returns.

The zero-shot result is evidence that the particular archived run produced positive simulated performance on a broad declared universe. It is not, by itself, evidence of universal generalization. The result depends on the data download, missing-data treatment, checkpoint, execution assumptions, and the absence of a parallel baseline table in the repository.

12 Forward Paper-Trading Simulation

The paper-trading program is more execution-aware than the training loop. It changes the model temperature from 0.1 to 0.5, applies a 30-percent maximum position cap, smooths weights with an exponential moving average factor of 0.3, targets 15-percent annualized volatility after at least twenty realized daily returns, limits desired turnover to 15 percent, and applies latency by trading toward the prior target. It also models 0.1-percent commission, 0.05-percent slippage, and a 90-percent randomized fill rate.

For target weights u_t and previous executed weights w_{t-1} , the desired trade size is computed as

$$q_t = \frac{1}{2} \|u_t - w_{t-1}\|_1. \quad (41)$$

If q_t exceeds the turnover limit $q_{\max} = 0.15$, the target move is scaled. If the fill fraction is f_t , the executed weights are

$$w_t = w_{t-1} + f_t(\tilde{w}_t - w_{t-1}), \quad (42)$$

where $\tilde{w}_t$ is the turnover-limited target. The linear transaction cost is

$$\text{Cost}_t = (c_{\text{comm}} + c_{\text{slip}})q_t. \quad (43)$$

The net return is gross portfolio return minus this cost. The CSV contains 413 realized records. As shown in Table 7, the independent recomputation agrees with the rounded paper-trading log. The strategy ends at equity approximately 1.463, corresponding to a cumulative return of 46.30 percent. The deepest wealth-relative drawdown is approximately negative 7.97 percent.

Figure 2 shows a more stable-looking equity path over the shorter window, with drawdowns concentrated around

Table 7: Forward paper-trading metrics from the repository.

Metric	Logged	CSV recomputation
Total return	46.30%	46.2954%
Sharpe ratio	2.223	2.2229
Sortino ratio	3.389	3.3889
Maximum drawdown	-7.97%	-7.9732%
Profit factor	1.44	1.4401
Average turnover	4.32%	4.3240%
Rows	413	413

early 2023, autumn 2023, and spring 2024. The stronger Sharpe and smaller drawdown relative to the zero-shot run should not be interpreted as proof that post-processing always improves performance. The horizons, asset mixes, and market regimes differ, and the paper-trading program uses stochastic fill fractions.

13 Execution Layer as a Mathematical Policy

The repository makes a useful distinction between a signal policy and an execution policy. The signal policy maps a normalized feature window to raw softmax weights. The execution policy maps those raw weights, previous holdings, and market-history estimates to executed weights and net returns.

Let s_t denote the raw model scores. The paper-trading temperature produces

$$w_t^{\text{raw}} = \text{softmax}(s_t/0.5). \quad (44)$$

After the per-asset cap and renormalization, EMA smoothing gives

$$w_t^{\text{smooth}} = 0.3w_t^{\text{cap}} + 0.7w_{t-1}^{\text{smooth}}. \quad (45)$$

When at least twenty daily returns have been observed, the annualized realized volatility estimate is

$$\hat{\sigma}_t = \sqrt{252} \text{Std}(R_{t-19}, \dots, R_t). \quad (46)$$

The code scales the smoothed weights by $0.15/\hat{\sigma}_t$ when the estimate is positive. It then normalizes only if the resulting sum exceeds one. This means the paper-trading layer can reduce gross exposure below one during high volatility; it is not simply another simplex projection.

With latency enabled, the target used for execution is the previous day’s target when one exists. This introduces a one-step delay between signal generation and target execution. Randomized fills then make the realized path stochastic unless the NumPy random state is controlled. These details are essential when interpreting the paper-trading CSV as an execution simulation rather than as a deterministic mathematical backtest.

14 Comparative Interpretation

Table 8 places the three available result classes side by side. The training-test row is generated by the original six-asset training script. The zero-shot row is a cost-aware evaluation on 98 assets over 2017–2024. The

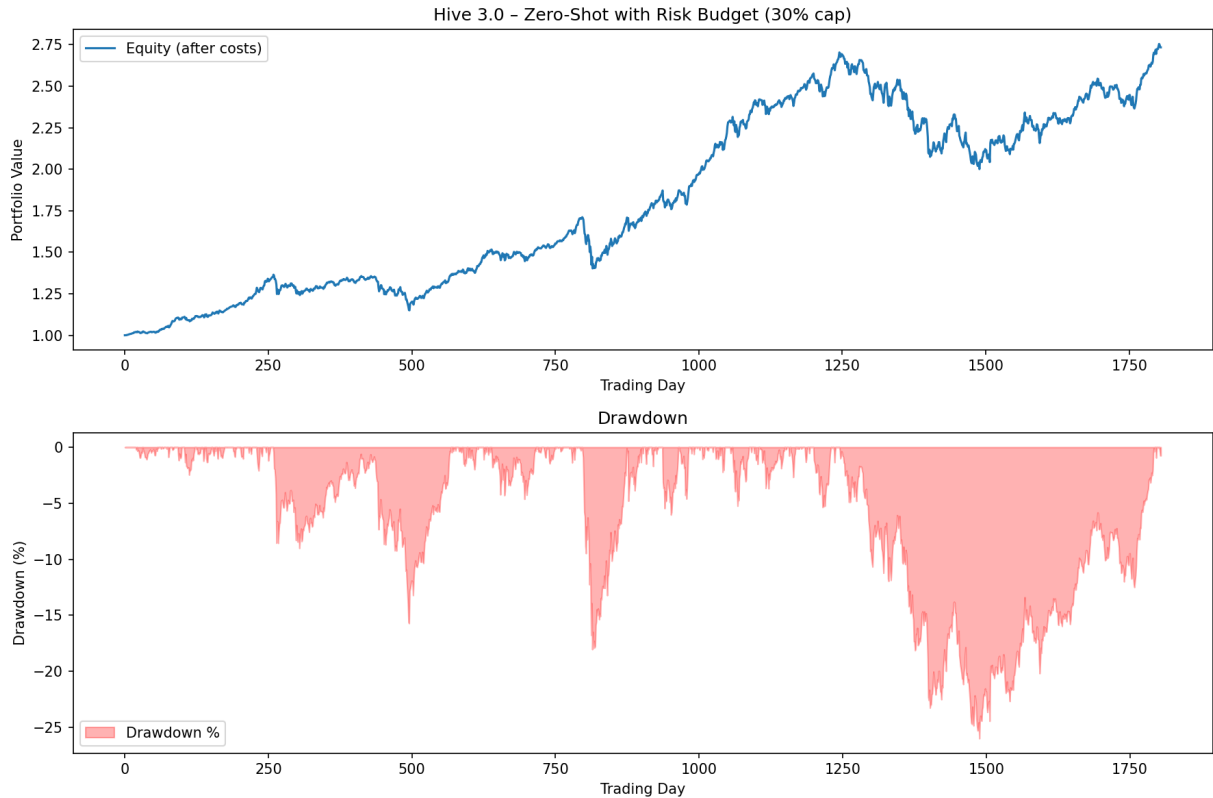


Figure 1: Repository zero-shot equity curve and drawdown. The plot is the supplied `hive_zeroshot.png`; the backtest labels the equity series as after costs and applies a 30-percent per-asset risk budget.

Table 8: Comparison of archived Hive evaluation outputs.

Evaluation	Assets	Period	Return	Sharpe	Max DD	Turnover	
Training script test	6	Chronological held-out block	—	0.620	23.70% ^a	—	^a The training script prints 0.2370 as its cumulative-return drawdown statistic; its definition differs from the wealth-relative percentage drawdown used by the backtest and paper-trading programs.
Zero-shot backtest	98	2017–2024	173.43%	1.146	-26.00%	3.74%	
Paper trading	98	2023–2024	46.30%	2.223	-7.97%	4.32%	

forward row is a cost-aware paper simulation on a declared 98-asset universe over 2023–2024.

The results are compatible with the repository’s transfer hypothesis: a model trained on six volatility products generated positive archived performance on two broad unseen evaluation runs. However, the evidence is not a randomized experiment. The repository provides one checkpoint, one primary training run, no ablation table, no benchmark portfolio, no confidence intervals, and no independent data vendor. The paper therefore reports the results faithfully while avoiding a causal claim that the architecture alone produced them.

15 Threats to Validity and Reproducibility

Several threats should be considered before treating the archived results as confirmatory evidence. First, the model checkpoint and logs represent one stochastic training path. The optimizer uses shuffled batches, and the paper-trading fills are randomized. Multiple seeds and confidence intervals are needed to quantify variability. Second, the data are downloaded at runtime through

‘yfinance’. Historical data can change because of vendor revisions, ticker survivorship, delistings, corporate-action adjustments, and calendar differences. A reproducible release should archive raw prices and record exact package versions.

Third, the 98-asset universe is broad but not a formal benchmark. The code fills failed downloads with a constant value in some pathways. Such behavior can create degenerate assets and should be audited in every re-run. The declared universe also contains instruments with different liquidity, trading hours, and structural exposures.

Fourth, the selection protocol is not fully nested. The best validation checkpoint is selected from the training script, after which the same saved state is used in later backtest programs. This is reasonable for a deployment simulation, but performance claims still depend on the checkpoint-selection history and should be evaluated with a fully frozen research boundary.

Fifth, the execution model is simplified. Commission and slippage are linear in trade size, fill rates are synthetic, and market impact is absent. The model does not

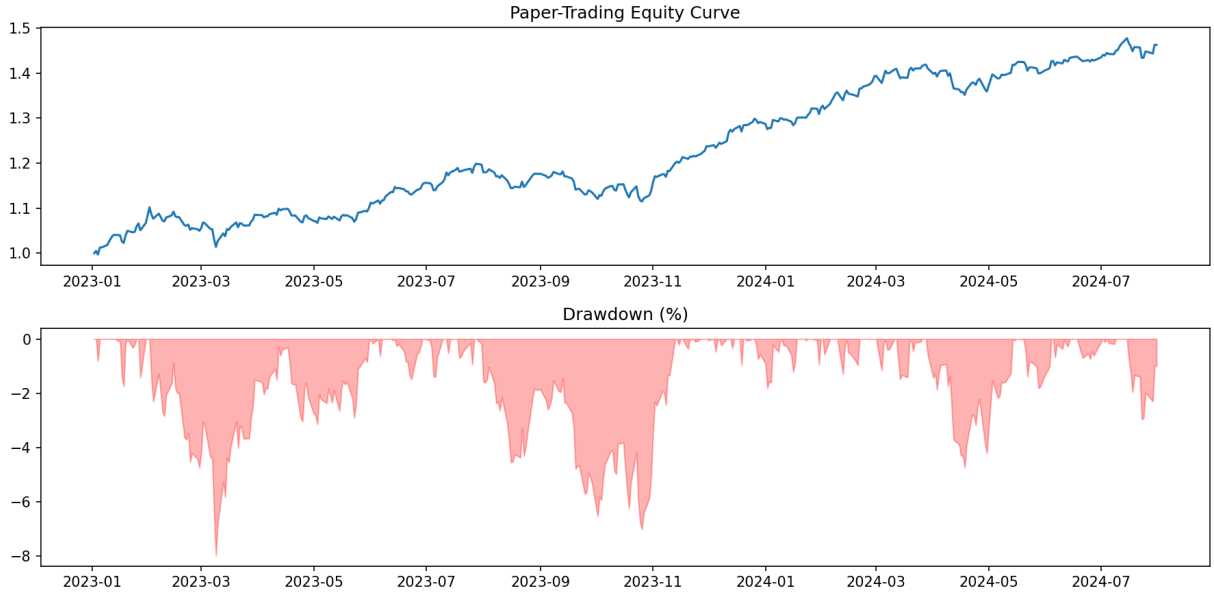


Figure 2: Repository forward paper-trading equity curve and drawdown. The shorter window ends in July 2024 and uses the execution adjustments listed in Table 6.

represent bid–ask spread dynamics, order queue position, borrow costs, financing, or asset-specific liquidity. The paper-trading simulation is more realistic than frictionless return multiplication, but it remains a simulation.

Finally, the repository README includes a claim that post-processing improves a raw signal from Sharpe 0.245 and 34-percent daily turnover. Because those raw outputs are not present in the archived CSV files used here, this paper treats that statement as an unverified repository claim and does not use it as a primary result.

16 Discussion and Limitations

The architecture separates shared computation from independent state. Assets use the same latent strategy vocabulary, but no hidden state is passed from one asset to another. Cross-asset information enters through the final competition among asset scores and through the global reductions used for strategy attention.

The Sharpe objective is batch-dependent. The gradient of one sample’s contribution depends on the other returns in the batch through the mean and standard deviation. Batch composition can therefore influence optimization even when the underlying windows are unchanged.

The model omits transaction costs, slippage, market impact, turnover, liquidity limits, taxes, leverage constraints, and financing. It also does not penalize changes in allocation between successive forecast origins. A production extension could add a turnover cost such as

$$\lambda_{\text{turn}} \|w_t - w_{t-1}\|_1. \quad (47)$$

The expanding scaler is causal with respect to the forecast origin but includes the current window in its scale estimate. A strict study should use training-fold-only preprocessing, walk-forward validation, multiple random seeds, baselines, confidence intervals, and cost-aware reporting. Since no completed logs were supplied, this paper deliberately does not claim a Sharpe ratio, return,

or drawdown result.

17 Reproducibility and Research Disclaimer

The source and model description are available at github.com/ak495867/Hive. Reproduction requires archiving the exact downloaded data, package versions, random seeds, model checkpoint, and complete logs because the script downloads market data at runtime.

HIVE is intended for educational and research purposes. It is not financial, investment, or trading advice. Historical or simulated performance does not guarantee future results. The implementation does not establish profitability, robustness, or suitability for live trading.

18 Conclusion

This paper formalizes HIVE as a differentiable policy over the portfolio simplex. Four engineered close-price features are normalized and presented as twenty-step sequences. Each asset is processed by a shared bank of thirty leaky recurrent strategy states. The states interact through adaptive five-node ring neighborhoods, where a learned compatibility function selects neighboring information and a multiplicative transform creates non-linear coupling. Local readouts and a batch-contextual strategy-attention mechanism produce one score per asset. A temperature softmax maps those scores to a fully invested allocation, and a Sharpe-based objective trains the complete pipeline end to end.

The architecture is compact and interpretable at the level of information pathways. It remains a research prototype. The next empirical step is a controlled evaluation with frozen preprocessing, walk-forward validation, transaction costs, turnover accounting, baselines, and repeated seeds. Only then can the model’s economic value be assessed.

References

- [1] A. Varma, “Hive: A Locally Coupled Neural Portfolio Allocator,” GitHub repository, 2026. <https://github.com/ak495867/HIve>.
- [2] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” arXiv:1412.6980, 2014. <https://arxiv.org/abs/1412.6980>.
- [3] A. Vaswani et al., “Attention Is All You Need,” arXiv:1706.03762, 2017. <https://arxiv.org/abs/1706.03762>.
- [4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [5] PyTorch Documentation, “Neural network modules, softmax, Adam, and gradient clipping.” <https://pytorch.org/docs/stable/>.